Scheduler Backend – Endpoint Overview

1) POST `/interview` – Create Interview

- **Auth**: Required
- **What it does**: Schedules an interview for a match profile, creates a calendar event (Google/Teams), and stores an event record for later reschedule/cancel.
- **Key inputs (typical)**:
  - `matchProfile` (string id)
  - `service` (`"google-calendar"` | `"teams"`)
  - `start_time`, `end_time` (ISO strings)
  - optional metadata (title/description/timezone fields, etc.)
- **High-level output**: `201 CREATED` with meeting payload from the scheduling system.

Scenarios

- **Scenario A – Schedule using Google Calendar**
  - `service = "google-calendar"` and a valid `matchProfile` → calendar event is created in Google Calendar and stored.
- **Scenario B – Schedule using Microsoft Teams**
  - `service = "teams"` and a valid `matchProfile` → calendar event is created in Outlook/Teams and stored.
- **Scenario C – Invalid or missing matchProfile**
  - `matchProfile` missing, malformed, or non-existent → request fails with “match profile not found”.
- **Scenario D – Not authenticated**
  - Missing/invalid `Authorization` header → request rejected with unauthorized error.
- **Scenario E – Downstream scheduling system fails**
  - External scheduling system returns 4xx/5xx → error status and message bubbled back to caller.

---

2) PUT `/interview/:id/reschedule` – Reschedule Interview

- **Auth**: Required
- **What it does**: Changes the interview time window and updates the stored calendar event + event record.
- **Key inputs (typical)**:
  - `:id` (meeting/event id)
  - `start_time`, `end_time`, `timezone`, `candidate_timezone`
  - optional: `title`, `description`, `meeting_link`, `stage`
- **High-level output**: `200 OK` (often `data: {}`).

Scenarios

- **Scenario A – Successful reschedule**
  - Interview exists, caller is authenticated, and new times are valid → calendar event and stored event record are updated.
- **Scenario B – Interview/event not found**
  - `:id` does not map to an existing event → operation fails as “not found” or internal error depending on context.
- **Scenario C – Invalid date window**
  - `start_time`/`end_time` missing or invalid (end before start, bad format) → request fails with validation or internal error.
- **Scenario D – Not authenticated**
  - Missing/invalid token → unauthorized error.
- **Scenario E – Calendar provider error**

- Calendar reschedule call fails (e.g., token revoked) → internal error returned to client.

---

3) POST `/interview/:id/cancel` – Cancel Interview

- **Auth**: Required
- **What it does**: Cancels the interview and cancels the stored calendar event.
- **Key inputs (typical)**:
  - `:id` (meeting/event id)
  - optional body (e.g., `jobId` sometimes provided by callers)
- **High-level output**: `200 OK`.

Scenarios

- **Scenario A – Successful cancellation**
  - Interview exists, caller authenticated → meeting is canceled and calendar event is removed.
- **Scenario B – Interview/event not found**
  - `:id` does not map to an existing stored event → operation fails as "not found" or internal error depending on lookup result.
- **Scenario C – Not authenticated**
  - Missing/invalid token → unauthorized error.
- **Scenario D – Calendar provider error**
  - Cancel call to calendar provider fails → internal error returned to client.

---

2) POST `/user` – Request User Availability

- **Auth**: Required
- **What it does**: Creates/updates availability-request "user records" for multiple match profiles and returns a per-matchProfile success/error breakdown.
- **Key inputs (typical)**:
  - `matchProfiles` (array of ids)
  - `org_name`, `job_name`
  - `subject`, `body` (must include `{{availability_link}}`)
  - optional: `from`, `interviewerAvailability`, `interviewerAvailabilitySlots`, `availabilityTemplateId`
- **High-level output**: `201 CREATED` with:
  - `users`: map of created/updated records by matchProfile id
  - `errors`: array of failures (if any)

Scenarios

- **Scenario A – Single matchProfile request**
  - One valid matchProfile id → one user record created/updated, appears under `users` with no error.
- **Scenario B – Multiple matchProfiles with partial failures**
  - Some matchProfiles invalid or missing candidate email → valid ones appear in `users`, failures listed in `errors`.
- **Scenario C – Resend after failure**
  - Existing user has status "Not Started" or "Failed" → record is updated and re-used in response.
- **Scenario D – Missing template placeholder**
  - `body` does not contain `{{availability_link}}` → treated as a validation issue for each affected matchProfile (error entry).
- **Scenario E – Missing subject / org / job**
  - `subject`, `org_name`, or `job_name` missing or empty → error entry per affected matchProfile.
- **Scenario F – Not authenticated**
  - Missing/invalid token → unauthorized error and no users processed.

---

3) POST `/availability` – Create Availability

- **Auth**: Not required
- **What it does**: Candidate (or UI) submits availability slots for a user; creates or updates an availability record.
- **Key inputs (typical)**:
  - `user` (string id)
  - `timezone` (IANA string)
  - `availabilityData`: array of dates with slot ranges
  - optional: `customavailabilityData`, `description`
- **High-level output**: `201 CREATED` with the saved availability record.

Scenarios

- **Scenario A – First-time submission**
  - User exists, no prior availability → new availability record is created and returned.
- **Scenario B – Update existing availability**
  - User already has availability → existing record is updated with new data and returned.
- **Scenario C – Non-existent user**
  - `user` id does not resolve to a known user → "user not found" error.
- **Scenario D – Missing required fields**
  - `user`, `timezone`, or `availabilityData` missing/invalid → request fails with validation or internal error.

---

4) GET `/availability` – Get All Availabilities

- **Auth**: Not required
- **What it does**: Returns all availability records.
- **Key inputs**: none
- **High-level output**: `200 OK` with an array (can be empty).

Scenarios

- **Scenario A – Availabilities exist**
  - One or more records stored → array of availability objects returned.
- **Scenario B – No availabilities**
  - Database contains no availability records → returns empty array with `200 OK`.
- **Scenario C – Database error**
  - Underlying query fails → internal error returned.

---

5) GET `/availability/:id` – Get Availability by ID

- **Auth**: Not required
- **What it does**: Fetches one availability record by availability id.
- **Key inputs**: `:id`
- **High-level output**: `200 OK` if found; `404` if not found.

Scenarios

- **Scenario A – Record found**
  - Valid `:id` for an existing record → that record is returned with `200 OK`.
- **Scenario B – Record not found**
  - `:id` does not exist → "availability not found" error.
- **Scenario C – Invalid id / DB error**
  - Malformed id or database failure → internal error returned.

---

6) PUT `/availability/:id` – Update Availability

- **Auth**: Not required
- **What it does**: Updates an availability record by id.
- **Key inputs**:
  - `:id`
  - body can include `availabilityData`, `customavailabilityData`, `timezone`, `description`
- **High-level output**: `200 OK` if found; `404` if not found.

Scenarios

- **Scenario A – Update timezone or slots**
  - Valid `:id` and body → updated availability record returned with `200 OK`.
- **Scenario B – Record not found**
  - `:id` does not match any record → "availability not found".
- **Scenario C – Invalid body / DB error**
  - Bad payload or database update failure → internal error.

---

7) GET `/availability/matchProfile/:matchProfileId` – Get Availability by Match Profile

- **Auth**: Not required
- **What it does**: Fetches availability for a match profile and filters out expired slots using the requested timezone (default `UTC`).
- **Key inputs**:
  - `:matchProfileId`
  - optional query: `timezone`
- **High-level output**: `200 OK` with filtered availability data; `404` if not found.

Scenarios

- **Scenario A – Default timezone filter**
  - No `timezone` query provided → expired slots filtered using `UTC`; remaining slots returned.
- **Scenario B – Custom timezone filter**
  - `timezone` query provided (e.g., `Asia/Kolkata`) → expired slots filtered using that timezone.
- **Scenario C – All slots expired**
  - All slots are past in the chosen timezone → `availabilityData` comes back empty, but request still succeeds with `200 OK`.
- **Scenario D – Match profile not found**
  - No availability for given `:matchProfileId` → "availability not found".
- **Scenario E – Invalid timezone value**
  - Timezone parsing fails → internal error.

---

8) GET `/auth/validate/service` – Validate Service Integration

- **Auth**: Required
- **What it does**: Checks whether stored service integrations are still valid and returns a summary (valid vs revoked).
- **Key inputs**:
  - query: `email`
- **High-level output**: `200 OK` with validation summary; `400` if email missing.

Scenarios

- **Scenario A – Valid integrations**

    - Stored tokens for services (e.g., Google/Outlook) are still valid → services listed under “valid” with no revocations.
- **Scenario B – Some services revoked**
    - One or more stored tokens are no longer valid → those services appear under “revoked” and may be cleaned up; summary message guides user to re-authorize.
- **Scenario C – No integration set up**
    - No tokens stored for the given email → “integration not done” style message with `200 OK`.
- **Scenario D – Missing email query**
    - `email` not provided → `400` error indicating email is required.
- **Scenario E – Not authenticated**
    - Missing/invalid token → unauthorized error.

---

9) PUT `/scheduling/:meetingid/reschedule` – Reschedule Interview

- **Auth**: Required
- **What it does**: Changes the interview time window and updates the stored calendar event + event record.
- **Key inputs (typical)**:
    - `:id` (meeting/event id)
    - `start_time`, `end_time`, `timezone`, `candidate_timezone`
    - optional: `title`, `description`, `meeting_link`, `stage`
- **High-level output**: `200 OK` (often `data: {}`).

Scenarios

- **Scenario A – Successful reschedule**
    - Interview exists, caller is authenticated, and new times are valid → calendar event and stored event record are updated.
- **Scenario B – Interview/event not found**
    - `:id` does not map to an existing event → operation fails as “not found” or internal error depending on context.
- **Scenario C – Invalid date window**
    - `start_time`/`end_time` missing or invalid (end before start, bad format) → request fails with validation or internal error.
- **Scenario D – Not authenticated**
    - Missing/invalid token → unauthorized error.
- **Scenario E – Calendar provider error**
    - Calendar reschedule call fails (e.g., token revoked) → internal error returned to client.

---

10) POST `/scheduling/:meetingid/cancel` – Cancel Interview

- **Auth**: Required
- **What it does**: Cancels the interview and cancels the stored calendar event.
- **Key inputs (typical)**:
    - `:id` (meeting/event id)
    - optional body (e.g., `jobId` sometimes provided by callers)
- **High-level output**: `200 OK`.

Scenarios

- **Scenario A – Successful cancellation**
    - Interview exists, caller authenticated → meeting is canceled and calendar event is removed.
- **Scenario B – Interview/event not found**
    - `:id` does not map to an existing stored event → operation fails as “not found” or internal error depending on lookup result.
- **Scenario C – Not authenticated**
    - Missing/invalid token → unauthorized error.
- **Scenario D – Calendar provider error**

- Cancel call to calendar provider fails → internal error returned to client.

---

11) POST /`scheduling/interview` – Create Interview

- **Auth**: Required
- **What it does**: Schedules an interview for a match profile, creates a calendar event (Google/Teams), and stores an event record for later reschedule/cancel.
- **Key inputs (typical)**:
  - `matchProfile` (string id)
  - `service` (`"google-calendar"` | `"teams"`)
  - `start_time`, `end_time` (ISO strings)
  - optional metadata (title/description/timezone fields, etc.)
- **High-level output**: `201 CREATED` with meeting payload from the scheduling system.

Scenarios

- **Scenario A – Schedule using Google Calendar**
  - `service = "google-calendar"` and a valid `matchProfile` → calendar event is created in Google Calendar and stored.
- **Scenario B – Schedule using Microsoft Teams**
  - `service = "teams"` and a valid `matchProfile` → calendar event is created in Outlook/Teams and stored.
- **Scenario C – Invalid or missing matchProfile**
  - `matchProfile` missing, malformed, or non-existent → request fails with “match profile not found”.
- **Scenario D – Not authenticated**
  - Missing/invalid `Authorization` header → request rejected with unauthorized error.
- **Scenario E – Downstream scheduling system fails**
  - External scheduling system returns 4xx/5xx → error status and message bubbled back to caller.

---

12) POST /`scheduling/availability` – Create Availability

- **Auth**: Not required
- **What it does**: Candidate (or UI) submits availability slots for a user; creates or updates an availability record.
- **Key inputs (typical)**:
  - `user` (string id)
  - `timezone` (IANA string)
  - `availabilityData`: array of dates with slot ranges
  - optional: `customavailabilityData`, `description`
- **High-level output**: `201 CREATED` with the saved availability record.

Scenarios

- **Scenario A – First-time submission**
  - User exists, no prior availability → new availability record is created and returned.
- **Scenario B – Update existing availability**
  - User already has availability → existing record is updated with new data and returned.
- **Scenario C – Non-existent user**
  - `user` id does not resolve to a known user → “user not found” error.
- **Scenario D – Missing required fields**
  - `user`, `timezone`, or `availabilityData` missing/invalid → request fails with validation or internal error